

MSD Homework 2, Problem 3

Nabin Oli (Caldwell University)

2026-06-12 22:39:29.520872

Contents

Description	2
Part A	2
Get and clean the raw data	3
Load the cleaned data	3
Your written answer	3
Join ngram year counts and totals	4
Plot the main figure 3a	4
Your written answer	5
Part C	5
Compare to the NGram Viewer	5
Your written answer	5
Part D	5
Plot the main figure 3a with raw counts	5
Part E	6
Plot the totals	6
Your written answer	7
Compute peak mentions	8
Compute half-lives	8
Plot the inset of figure 3a	8
Your written answer	9
Your written answer	9
Makefile	10

Description

This is a template for exercise 6 in Chapter 2 of *Bit By Bit: Social Research in the Digital Age* by Matt Salganik. The problem is reprinted here with some additional comments and structure to facilitate a solution.

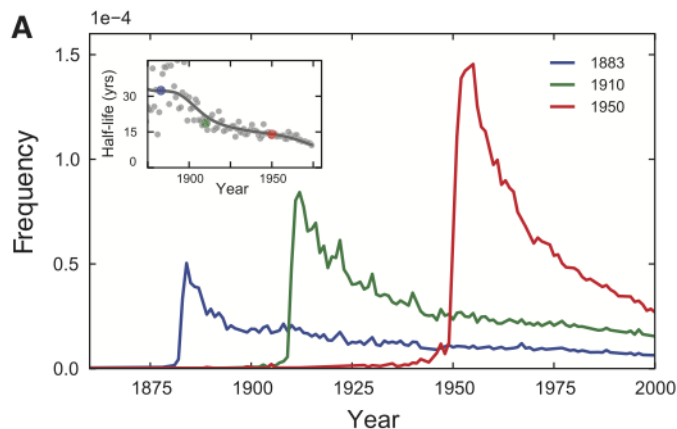
The original problem statement:

In a widely discussed paper, Michel and colleagues (2011) analyzed the content of more than five million digitized books in an attempt to identify long-term cultural trends. The data that they used has now been released as the Google NGrams dataset, and so we can use the data to replicate and extend some of their work.

In one of the many results in the paper, Michel and colleagues argued that we are forgetting faster and faster. For a particular year, say “1883,” they calculated the proportion of 1-grams published in each year between 1875 and 1975 that were “1883”. They reasoned that this proportion is a measure of the interest in events that happened in that year. In their figure 3a, they plotted the usage trajectories for three years: 1883, 1910, and 1950. These three years share a common pattern: little use before that year, then a spike, then decay. Next, to quantify the rate of decay for each year, Michel and colleagues calculated the “half-life” of each year for all years between 1875 and 1975. In their figure 3a (inset), they showed that the half-life of each year is decreasing, and they argued that this means that we are forgetting the past faster and faster. They used Version 1 of the English language corpus, but subsequently Google has released a second version of the corpus. Please read all the parts of the question before you begin coding.

This activity will give you practice writing reusable code, interpreting results, and data wrangling (such as working with awkward files and handling missing data). This activity will also help you get up and running with a rich and interesting dataset.

The full paper can be found [here](#), and this is the original figure 3a that you’re going to replicate:



Part A

Get the raw data from the Google Books NGram Viewer website. In particular, you should use version 2 of the English language corpus, which was released on July 1, 2012. Uncompressed, this file is 1.4GB.

Get and clean the raw data

Edit the `01_download_1grams.sh` file to download the `googlebooks-eng-all-1gram-20120701-1.gz` file and the `02_filter_1grams.sh` file to filter the original 1gram file to only lines where the ngram matches a year (output to a file named `year_counts.tsv`).

Then edit the `03_download_totals.sh` file to down the `googlebooks-eng-all-totalcounts-20120701.txt` and file and the `04_reformat_totals.sh` file to reformat the total counts file to a valid csv (output to a file named `total_counts.csv`).

Load the cleaned data

Load in the `year_counts.tsv` and `total_counts.csv` files. Use the `here()` function around the filename to keep things portable. Give the columns of `year_counts.tsv` the names `term`, `year`, `volume`, and `book_count`. Give the columns of `total_counts.csv` the names `year`, `total_volume`, `page_count`, and `book_count`. Note that column order in these files may not match the examples in the documentation.

```
year_table <- read_tsv(here('week3/ngrams', 'year_counts.tsv'),
                      col_names = c('term', 'year', 'volume', 'book_count'))

## Rows: 290916 Columns: 4
## -- Column specification -----
## Delimiter: "\t"
## dbf (4): term, year, volume, book_count
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

total_counts_table <- read_csv(here('week3/ngrams', 'total_counts.csv'),
                              col_names = c('year', 'total_volume', 'page_count', 'book_count'))

## Rows: 425 Columns: 4
## -- Column specification -----
## Delimiter: ","
## dbf (4): year, total_volume, page_count, book_count
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Your written answer

Add a line below using Rmarkdown's inline syntax to print the total number of lines in each dataframe you've created. There are total of 290916 in `year.csv` and there are 425 rows in `total.tsv` # Part B

Recreate the main part of figure 3a of Michel et al. (2011). To recreate this figure, you will need two files: the one you downloaded in part (a) and the "total counts" file, which you can use to convert the raw counts into proportions. Note that the total counts file has a structure that may make it a bit hard to read in. Does version 2 of the NGram data produce similar results to those presented in Michel et al. (2011), which are based on version 1 data?

Join ngram year counts and totals

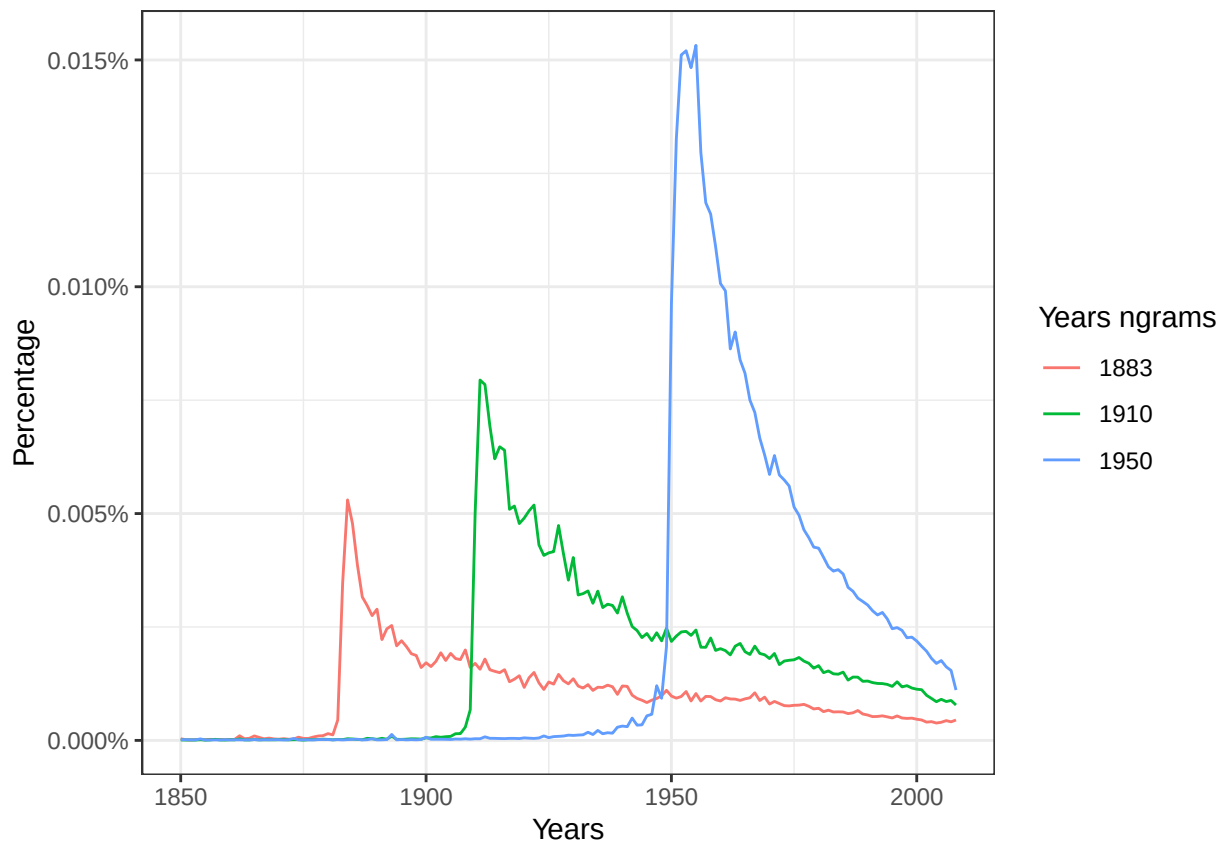
Join the raw year term counts with the total counts and divide to get a proportion of mentions for each term normalized by the total counts for each year.

```
Prop_table <- full_join(year_table, total_counts_table, by = "year") %>%  
mutate(prop = volume / total_volume)
```

Plot the main figure 3a

Plot the proportion of mentions for the terms “1883”, “1910”, and “1950” over time from 1850 to 2012, as in the main figure 3a of the original paper. Use the `percent` function from the `scales` package for a readable y axis. Each term should have a different color, it’s nice if these match the original paper but not strictly necessary.

```
Prop_table %>%  
filter(year <= 2012 & year >= 1850) %>%  
filter(term %in% c("1883", "1910", "1950")) %>%  
select(term, year, prop) %>%  
ggplot(aes(x = year, y = prop, color = as.factor(term))) +  
geom_line() +  
scale_y_continuous(labels = percent) +  
xlab("Years") +  
ylab("Percentage") +  
labs(color = "Years ngrams")
```



Your written answer

Write up your answer to Part B here. Yes the graph looks similar to that on figure 3a of Michel et al.(2011)

Part C

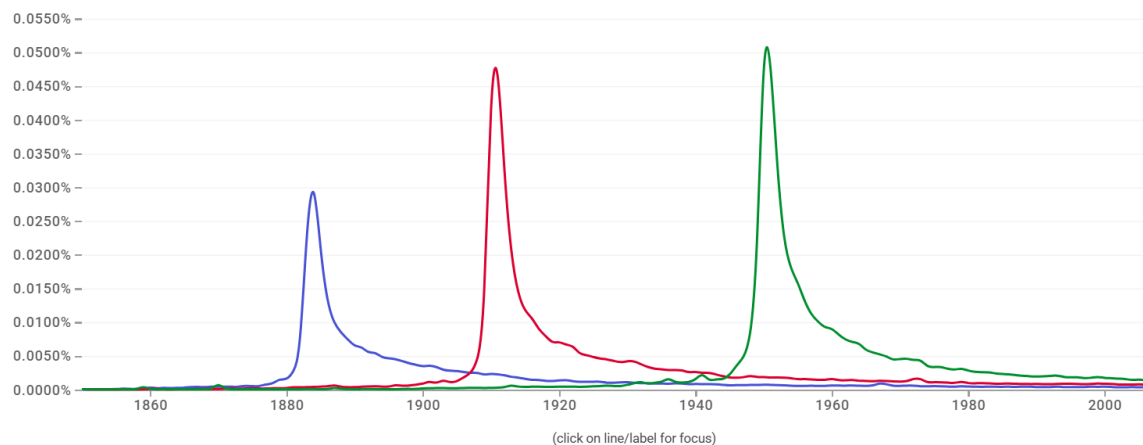
Now check your graph against the graph created by the NGram Viewer.

Compare to the NGram Viewer

Go to the ngram viewer, enter the terms “1883”, “1910”, and “1950” and take a screenshot.

Your written answer

Add your screenshot for Part C below this line using the `` syntax and comment on



similarities / differences.

- Overall shape of plot look similar, but there is mismatch in value of proportion probably because google Ngram viewer is based in updated data.
- Even peak year match in these graphs.

Part D

Recreate figure 3a (main figure), but change the y-axis to be the raw mention count (not the rate of mentions).

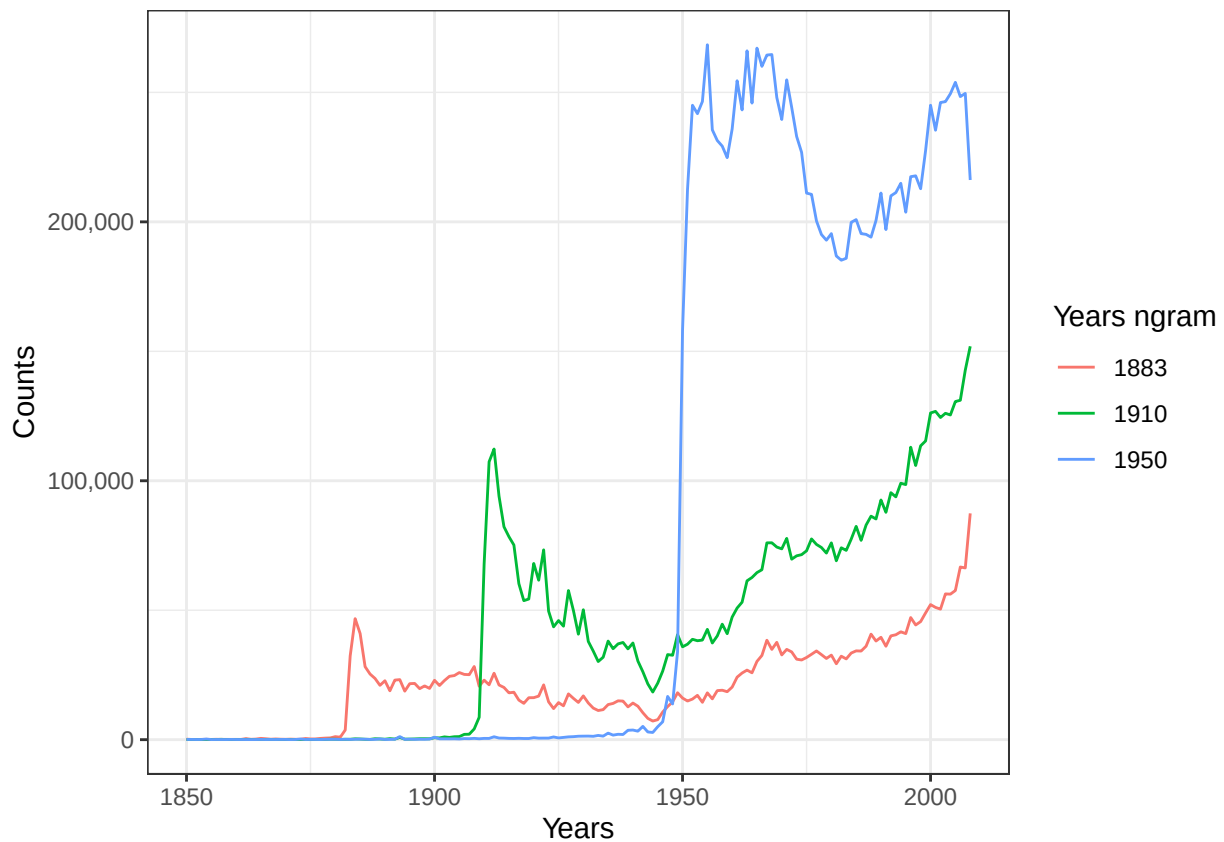
Plot the main figure 3a with raw counts

Plot the raw counts for the terms “1883”, “1910”, and “1950” over time from 1850 to 2012. Use the `comma` function from the `scales` package for a readable y axis. The colors for each term should match your last plot, and it’s nice if these match the original paper but not strictly necessary.

```

Prop_table%>%
  filter(year<=2012 & year >= 1850)%>%
  filter(term %in% c("1883", "1910", "1950"))%>%
  select(term, year, volume)%>%
  ggplot(aes(x = year, y=volume, color = as.factor(term)))+
  geom_line()+
  scale_y_continuous(labels = comma)+
  labs(x = "Years",
       y = "Counts",
       color = "Years ngram")

```



Part E

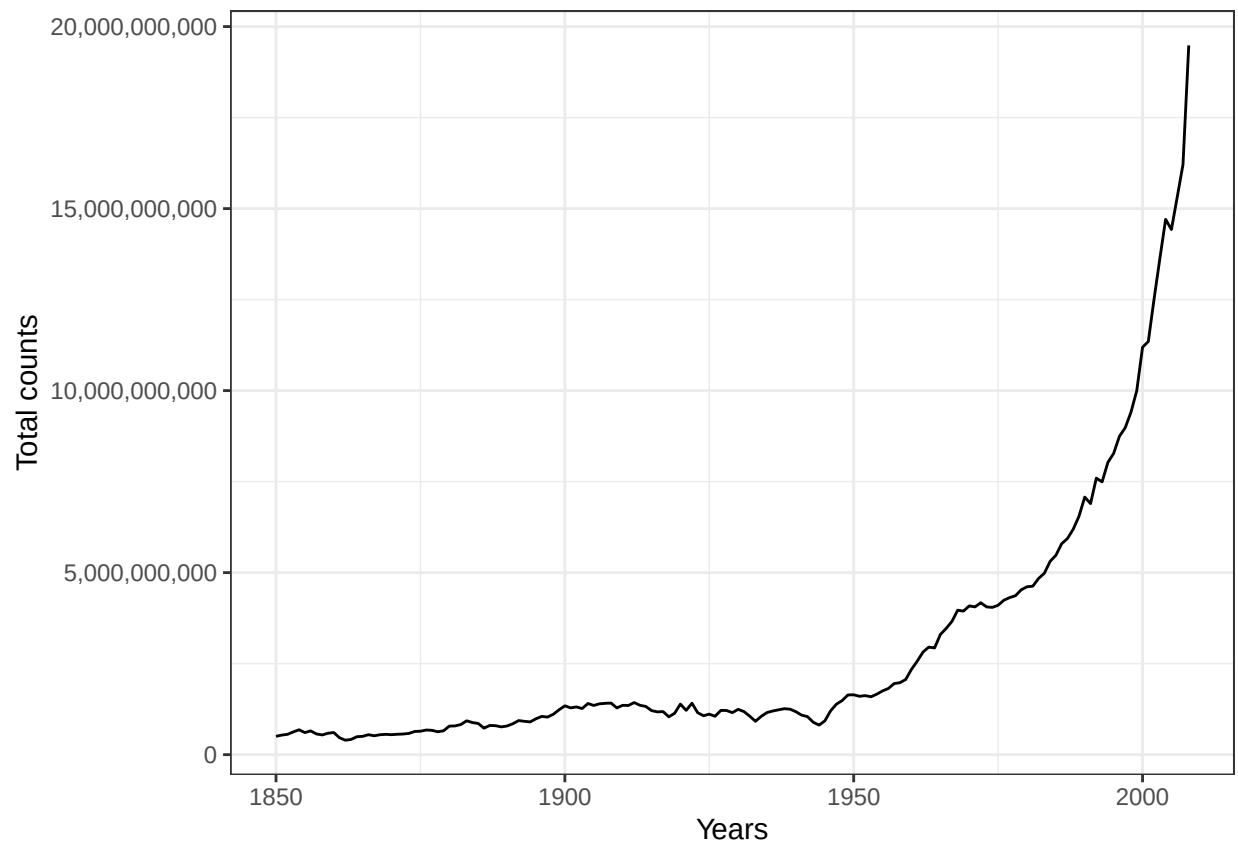
Does the difference between (b) and (d) lead you to reevaluate any of the results of Michel et al. (2011). Why or why not?

As part of answering this question, make an additional plot.

Plot the totals

Plot the total counts for each year over time, from 1850 to 2012. Use the `comma` function from the `scales` package for a readable y axis. There should be only one line on this plot (not three).

```
select(total_counts_table,year,total_volume)%>%
filter(year<=2012 & year>=1850)%>%
ggplot(aes(x=year,y=total_volume))+
geom_line()+
scale_y_continuous(labels = comma)+
labs(x="Years",
      y = "Total counts")
```



Your written answer

We might conclude that people are actually remembering more as year passes on by looking at part D as there is increase in the occurrence of terms with increasing year. But paper looks occurrence as a ratio to total counts of all ngrams that year and from plot-total, there is increase in number of total counts per year so we don't need to reevaluate any of the results of the paper.

Now, using the proportion of mentions, replicate the inset of figure 3a. That is, for each year between 1875 and 1975, calculate the half-life of that year. The half-life is defined to be the number of years that pass before the proportion of mentions reaches half its peak value. Note that Michel et al. (2011) do something more complicated to estimate the half-life—see section III.6 of the Supporting Online Information—but they claim that both approaches produce similar results. Does version 2 of the NGram data produce similar results to those presented in Michel et al. (2011), which are based on version 1 data? (Hint: Don't be surprised if it doesn't.)

Compute peak mentions

For each year term, find the year where its proportion of mentions peaks (hits its highest value). Store this in an intermediate dataframe.

```
max_peaks<-Prop_table%>%
  filter(year %in% c(1850:2012)& term %in% c(1875:1975))%>%
  group_by(term)%>%
  summarize(max_prop = max(prop))

max_peaks<-left_join(max_peaks,Prop_table,by = "term")%>%filter(year %in% c(1850:2012))%>%
  filter(max_prop==prop)%>%select(term,year,max_prop)
```

Compute half-lives

Now, for each year term, find the minimum number of years it takes for the proportion of mentions to decline from its peak value to half its peak value. Store this in an intermediate data frame.

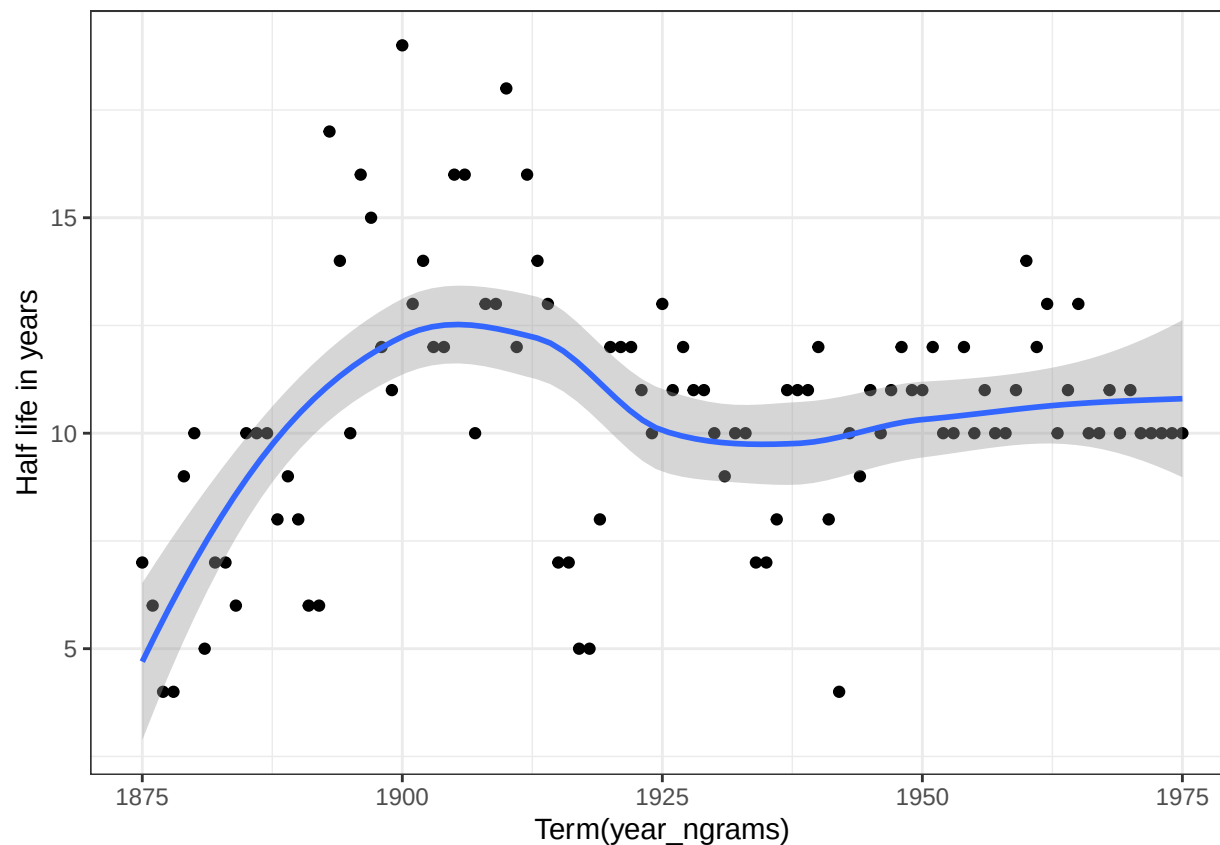
```
max_peaks<-rename(max_peaks,max_year=year)
Half_life<-full_join(Prop_table,max_peaks,by = "term")%>%
  filter((max_prop/prop)>=2 & year>max_year)%>%
  mutate(half_year = year - max_year)%>%
  group_by(term) %>%
  summarize(half_life = min(half_year))
```

Plot the inset of figure 3a

Plot the half-life of each term over time from 1850 to 2012. Each point should represent one year term, and add a line to show the trend using `geom_smooth()`.

```
Half_life%>%
  ggplot(aes(x=term,y=half_life))+
  geom_point()+
  geom_smooth()+
  labs(x="Term(year_ngrams)",
       y= "Half life in years")
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~
## x'
```

Your written answer

Write up your answer to Part F here. It doesn't have a same shape as in paper. There is no clear trend with increase in years. There are few outlier years that push and pull the line of fit. # Part G

Were there any years that were outliers such as years that were forgotten particularly quickly or particularly slowly? Briefly speculate about possible reasons for that pattern and explain how you identified the outliers.

Your written answer

Write up your answer to Part G here. Include code that shows the years with the smallest and largest half-lives.

```
Half_life%>%
  arrange(desc(half_life))%>%
  head()%>%kable()
```

term	half_life
1900	19
1910	18
1893	17

term	half_life
1896	16
1905	16
1906	16

These are top 6 most slowly forgotten years.

- 1900 is the top one because it is a start of new century.
- 1905 is the publication of special theory of relativity and it is widely mentioned in scientific literature.
- 1893 was chicago world fair and this year is generally mentioned when discussing about Tesla VS Edision Rivalry.

```
Half_life%>%
  arrange(half_life)%>%
  head()%>%kable()
```

term	half_life
1877	4
1878	4
1942	4
1881	5
1917	5
1918	5

These are top 6 quickly forgotten years.

- 1917 , 1918 and 1942 all fall between world wars. 1919 was when world war I ended and also 1939 was the year world war II started.

Makefile

Edit the **Makefile** in this directory to execute the full set of scripts that download the data, clean it, and produce this report. This must be turned in with your assignment such that running **make** on the command line produces the final report as a pdf file.